

Fjiffyldg API 参考手册

——高性能文件操作

工具库（版本 V1.0）

Du Jie (@ceepuka)

2026-1-12

目录

1. 概述	2
1.1 简介	2
1.2 功能特性	2
1.3 核心优势	2
2. 快速开始	3
2.1 环境要求	3
2.2 安装方法	3
2.3 使用示例	4
2.4 编译链接	5
3. 核心数据结构	6
3.1 文件处理器句柄	6
3.2 分层索引系统	6
3.3 状态管理系统	6
4. 文件加载与管理	8
4.1 创建与销毁	8
4.2 文件加载	10
4.3 状态查询	12
5. 行索引访问	13
5.1 行数查询	13
5.2 位置转换	14
5.3 长度获取	15
6. 数据读取操作	16
6.1 基本读取	16
6.2 行感知读取	17
6.3 大文件内存映射	19
7. 文件工具函数	22
7.1 文件操作	22
7.2 文本编码检测	23
7.3 编码字符统计	25
8. C++接口封装	26
8.1 C++类接口	26
8.2 C++使用示例	27
9. 高级功能	28
9.1 异步扫描控制	28
9.2 重新扫描功能	29
9.3 性能优化指南	30
附录 A: API 速查表	32
附录 B: 版本历史	36

1. 概述

1.1 简介

Fjiffyldg（高性能文件操作工具库）是一个完全自包含的 C/C++库，专注于提供极速启动和高效处理的文件数据操作能力。库采用创新的扫描并发架构，将核心数据处理与辅助索引构建分离。

核心设计理念：

- 数据处理优先：加载、读取、文本检查等操作即时可用
- 扫描并发执行：行索引在后台构建，不阻塞数据操作
- 内存智能管理：按需加载与分层索引相结合
- 实用工具完备：覆盖文件操作常见需求

1.2 功能特性

核心数据处理

- 即时数据访问：启动后 10ms 内即可读取文件数据
- 智能加载系统：数据按访问请求动态加载，内存高效
- 多编码文本处理：完整的 ASCII/UTF-8 检测与验证工具

辅助索引支持

- 行索引构建：后台扫描线程建立行位置信息
- 分层索引设计：前 100 万行直接索引，超出行数智能管理
- 行感知操作：精确的行内数据读取与控制

文件操作工具集

- 原子文件操作：安全的复制、保存、追加、合并
- 独立映射系统：完整文件内存映射（与加载系统分离）
- 跨平台原生：Windows、Linux、macOS，无外部依赖

技术规格

- 接口设计：纯 C API + 现代 C++ RAII 封装
- 状态管理：加载状态与扫描进度独立监控
- 资源安全：显式清理与自动管理结合

1.3 核心优势

与传统方案对比

传统文件库：加载(阻塞) → 扫描(阻塞) → 完成 → 开始操作
Fjiffyldg：启动扫描 → 立即操作 ← 后台索引构建
 ↳ 数据按需加载

关键优势

维度	传统方案	Fjiffyldg	用户价值
----	------	-----------	------

启动响应	秒级等待	<10ms 操作	即时可用
操作体验	扫描期间卡顿	零阻塞操作	流畅体验
内存效率	预加载占用大	按需加载	大文件友好
功能完备	常需组合多库	一体化工具集	开发简便

技术创新

- 无依赖实现：完全自包含，部署简单
- 状态分离设计：操作状态与索引进度解耦
- 精准控制：行边界约束，避免意外数据跨越
- 编码智能：完整与截断文本的双重处理机制

适用场景：

- 需要即时文件预览的应用
- 大日志文件的实时监控
- 文本编辑器和开发工具
- 数据清洗与转换流程
- 内存受限环境的文件处理

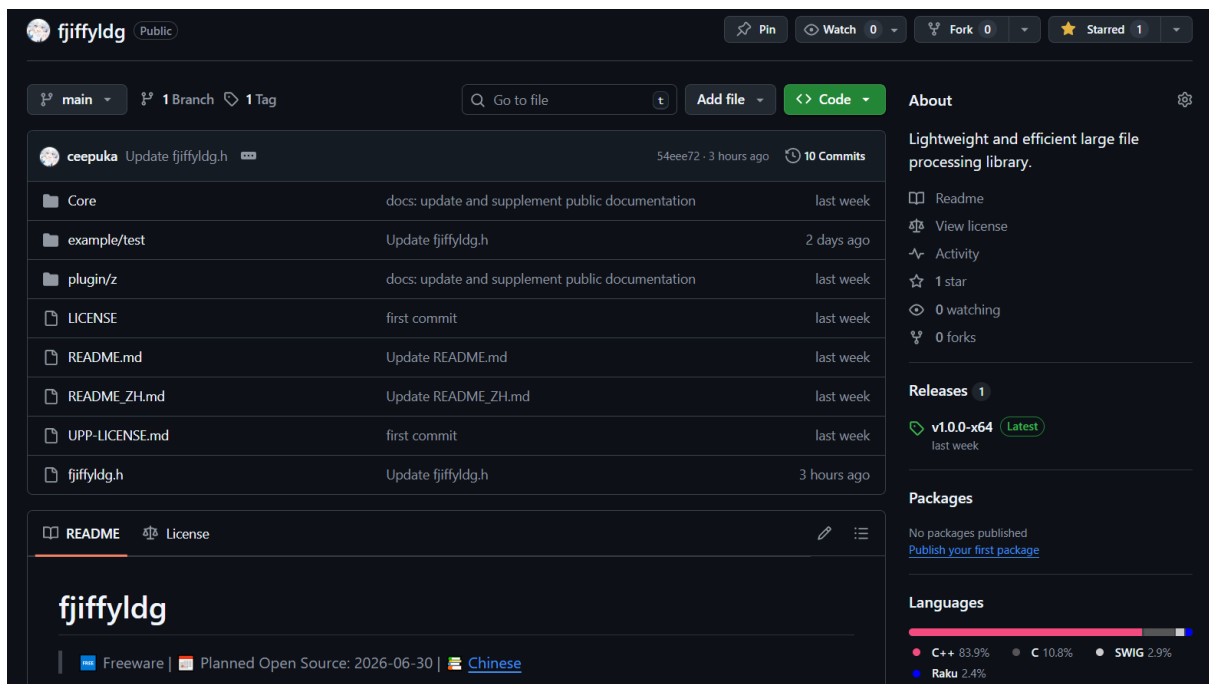
2. 快速开始

2.1 环境要求

系统平台	编译器（C++17 标准支持）	备注
Windows10 及以上	MSVC 2019+ 或 MinGW	
Linux	GCC 9.0+ 或 Clang 10+	Ubuntu 18.04+ / CentOS 8+ / 其他主流发行版
macOS 10.15 及以上	Clang 10+	

2.2 安装方法

从 GitHub 仓库 <https://github.com/ceepuka/fjiffyldg> 获取接口头文件及发布的静态库版本，并直接将文件集成到你的项目中即可。



2.3 使用示例

C 语言使用示例

```
#include "fjiffyldg.h"
#include <stdio.h>
int main() {
    // 创建句柄
    fjiffyldg_ptr fm = fjiffyldg_create();

    // 加载并扫描文件
    int result = LoadAndScanFile(fm, "example.txt");
    if (result != 0) {
        printf("Failed to load file\n");
        return -1;
    }

    // 获取文件行数
    long long lineCount = GetFileLineCount(fm);
    printf("Total lines: %lld\n", lineCount);

    // 读取第一行
    long long pos = GetFileLinePos(fm, 0);
    unsigned int len = 0;
```

```
const char* data = ReadFileData(fm, pos, &len);
if (data) {
    printf("Line 1: %.*s\n", len, data);
}

// 清理资源
fjiffyldg_clear(fm);
return 0;
}
```

C++ 语言使用示例

```
#include "fjiffyldg.h"
#include <iostream>
int main() {
    // 创建对象
    Fjiffyldg::Fjiffyldg fm;

    // 加载并扫描文件
    int result = LoadAndScanFile(fm.GetFjiffyldgHandle(), "example.txt");
    if (result != 0) {
        std::cerr << "Failed to load file" << std::endl;
        return -1;
    }

    // 获取行数
    long long lineCount = GetFileLineCount(fm.GetFjiffyldgHandle());
    std::cout << "Total lines: " << lineCount << std::endl;

    // 自动清理资源 (RAII)
    return 0;
}
```

2.4 编译链接

Fjiffyldg 非常容易从源码级别构建软件项目。

例如：

GCC

```
gcc -g [源文件] -lfjiffyldg -l[必要的系统库]... -lstdc++ -o [可执行文件]
```

Clang

```
clang -g [源文件] -lfjiffyldg -l[必要的系统库]... -lc++ -o [可执行文件]
```

或者直接使用 IDE（MSVC 等）更简单。

系统库依赖项说明:

平台	依赖项	提示
Windows	"kernel32 user32 ole32 oleaut32 uuid ws2_32 advapi32 shell32 winmm mpr"	一般手动添加"winmm mpr"
Linux	"pthread dl rt"	具体系统架构要求"execinfo posix4 nsl"等
macOS	"pthread"	

3. 核心数据结构

3.1 文件处理器句柄

```
typedef struct fjiffyldg_t * fjiffyldg_ptr;
```

- 功能: 所有文件操作的统一入口点
- 资源: 封装文件访问所需全部状态和资源
- 管理: 必须配对使用 fjiffyldg_create()和 fjiffyldg_clear()
- 线程: 单个句柄不支持并发访问

3.2 分层索引系统

设计原理

- 性能优先区: 前 100 万行使用直接索引, O(1)访问
- 内存优化区: 超过 100 万行使用离散块索引, O(log n)访问
- 平滑过渡: 根据行数自动选择索引策略

索引选择逻辑

```
// 伪代码说明
if (行号 < 1000000) {
    使用直接索引数组;
} else {
    使用离散块索引系统;
}
```

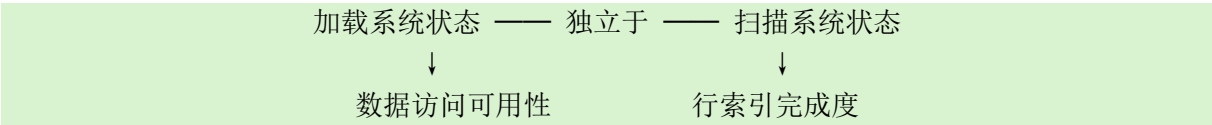
3.3 状态管理系统

核心状态

- 加载状态: 文件数据加载是否成功 (GetFileIsLoaded()查询)
- 扫描进度: 行索引构建完成程度 (GetFileLineCount()查询)

- 缓存状态：最近访问位置（优化连续访问性能）

状态分离设计



状态查询机制

双系统独立状态：

```
// 加载系统状态查询
int load_status = GetFileIsLoaded(fm);
// 0: 加载系统正常
// 非 0: 加载系统错误

// 扫描系统状态查询
long long scan_status = GetFileLineCount(fm);
// >0: 扫描完成，返回总行数
// =0: 扫描未开始
// <0: 扫描进行中
```

典型状态组合

GetFileIsLoaded	GetFileLineCount	系统状态
0	>0	完全就绪
0	0	就绪但未扫描
0	<0	运行中（扫描进行）
非 0	任意	系统故障

使用示例

基础使用模式

```
// 创建处理器
fjiffyldg_ptr fm = fjiffyldg_create();
if (!fm) {
    fprintf(stderr, "创建失败\n");
    return 1;
}

// 启动并发处理系统
int result = LoadAndScanFile(fm, "data.txt");
if (result == 0) {
    printf("系统启动成功，可立即操作\n");

    // 检查扫描进度（可选）
    long long scan_state = GetFileLineCount(fm);
    if (scan_state < 0) {
        printf("扫描进行中...\n");
    }
}
```

```
}

// 立即开始数据操作
unsigned int len = 1024;
const char* data = ReadFileData(fm, 0, &len);
if (data) {
    printf("成功读取 %u 字节\n", len);
}

} else {
    printf("启动失败, 错误码: %d\n", result);
}

// 清理资源
fjiffyldg_clear(fm);
```

高级使用模式

```
// 模式 1: 编辑器场景 (立即响应)
LoadAndScanFile(fm, file);
// 不等待扫描, 立即提供编辑功能
// 扫描在后台进行, 不影响用户体验

// 模式 2: 分析工具场景 (需要完整索引)
LoadAndScanFile(fm, file);
WaitFileScanTaskFinished(fm); // 等待扫描完成
// 现在有完整的行索引, 可进行批量分析

// 模式 3: 查看器场景 (仅需数据)
// 考虑使用 LoadFileOnly() 替代
```

4. 文件加载与管理

4.1 创建与销毁

fjiffyldg_create

```
FJIFFYLDG_API fjiffyldg_ptr fjiffyldg_create(void);
```

C 语言专用: 创建文件处理器实例。失败返回 NULL (内存不足)。

重要限制: C++ 代码禁止直接使用, 应通过 `Fjiffyldg::Fjiffyldg` 类自动管理。

返回值:

- 成功: 返回有效句柄指针
- 失败: 返回 NULL (通常内存不足)

示例:

```
fjiffyldg_ptr fm = fjiffyldg_create();
if (!fm) {
    fprintf(stderr, "错误: 无法创建文件处理器\n");
    return 1;
}
```

fjiffyldg_clear

FJIFFYLDG_API void fjiffyldg_clear(fjiffyldg_ptr fm);

C 语言专用: 释放处理器所有资源 (包括映射)。必须与 fjiffyldg_create() 配对使用。

C++ 替代: 通过 Fjiffyldg::Fjiffyldg 析构函数自动清理。

参数:

- fm: 要清理的文件处理器句柄

重要规则:

- 必须与 fjiffyldg_create() 配对使用
- 调用后句柄立即失效, 不应再次使用
- C++ 用户通过类析构函数自动管理

头文件中的限制声明

```
// C++ 代码中禁止直接使用
#ifdef __cplusplus
FJIFFYLDG_API fjiffyldg_ptr fjiffyldg_create(void);
FJIFFYLDG_API void fjiffyldg_clear(fjiffyldg_ptr fm);
#endif
```

使用规范对比

语言	创建方式	清理方式	线程安全
C 语言	fjiffyldg_create()	fjiffyldg_clear()	需手动配对
C++	Fjiffyldg::Fjiffyldg fm;	自动析构	RAII 保障

C 语言使用示例

```
// 仅 C 语言可使用
fjiffyldg_ptr fm = fjiffyldg_create();
if (!fm) {
    fprintf(stderr, "创建失败\n");
    return 1;
}
```

```
// ... 使用 ...
```

```
fjiffyldg_clear(fm); // 必须显式清理
```

C++使用规范

```
// 正确：使用 C++类
```

```
Fjiffyldg::Fjiffyldg fm; // 自动构造
```

```
// ... 使用 ...
```

```
// 自动析构，无需手动清理
```

```
// 错误：直接调用 C 函数（编译错误）
```

```
// fjiffyldg_ptr fm = fjiffyldg_create(); // 禁止！
```

```
// fjiffyldg_clear(fm); // 禁止！
```

设计目的：

1. 强制 **RAII**：C++代码必须使用类封装，确保异常安全
2. 避免误用：防止 C++代码手动管理资源导致泄漏
3. 接口清晰：明确区分 C 和 C++的使用范式

4.2 文件加载

LoadAndScanFile - 核心加载函数

```
FJIFFYLDG_API int LoadAndScanFile(fjiffyldg_ptr fm, const char *name);
```

功能概述

启动 **Fjiffyldg** 的智能并发处理系统，采用扫描优先、加载后置的架构设计，为用户提供即时响应和后台高效处理的完美结合。

核心技术架构：

调用 **LoadAndScanFile()**

↓

[阶段 1：扫描系统优先启动]

- └─ 创建独立文件映射（专用于扫描线程）
- └─ 启动后台扫描线程
- └─ 开始字符级检查和行索引构建
- └─ 验证扫描线程成功运行

↓

[阶段 2：数据加载初始化]

- └─ 利用 **Ultimate++** 高效机制加载首数据块
- └─ 建立按需加载触发系统
- └─ 确保 **ReadFileData** 等 API 可立即使用

↓

[阶段 3: 立即返回用户控制]
└─ 返回 0 (表示扫描启动成功 + 首块加载完成)
└─ 系统进入完全并发工作模式

设计优势

优势	技术原理	用户价值
即时响应	扫描优先启动，不等待完整加载	<10ms 即可开始操作
真正并发	扫描与加载线程完全分离	扫描不阻塞数据读取
内存高效	按需分块加载机制	大文件处理无压力
架构优雅	职责分离，各司其职	系统稳定可靠

返回值含义:

返回值	含义	系统状态
0	成功启动	扫描线程运行，首块加载完成
-1	文件不存在	系统不可用
1	文件不可访问	系统不可用
2	文件流错误	系统不可用
3	内存映射错误	系统不可用

示例:

```
int result = LoadAndScanFile(fm, "large_data.log");
if (result == 0) {
    printf("系统启动成功，可立即读取数据\n");

    // 此时:
    // 1. 首数据块已加载
    // 2. 扫描线程在后台运行
    // 3. 可立即调用 ReadFileData 等函数
} else {
    printf("启动失败，错误码: %d\n", result);
}
```

适用场景推荐

- ✔ 大文件处理: 内存友好，按需加载
- ✔ 交互式应用: 即时响应，不卡界面
- ✔ 长时间扫描: 后台执行，用户无感
- ✔ 复杂编码文件: 并发处理，效率更高

注意事项

- 线程安全: 单个句柄非线程安全，多线程访问需要外部同步
- 资源管理: 必须配对使用 fjiffyldg_create 和 fjiffyldg_clear
- 指针安全: ReadFileData 返回只读指针，禁止修改或释放

4. 状态依赖：某些功能需要扫描完成（如准确的行数统计）

总结：LoadAndScanFile 是 Fjiffyldg 的核心入口，采用创新的扫描优先、并发执行架构，在保证功能完整性的同时，提供了传统库无法比拟的响应速度和用户体验。

LoadFileOnly - 简化加载函数

```
FJIFFYLDG_API int LoadFileOnly(fjiffyldg_ptr fm, const char *name);
```

功能：仅加载文件内容，而不构建行索引。

适用场景：

- 只需要文件原始数据
- 不需要按行访问功能
- 对启动速度有极致要求

与 LoadAndScanFile 对比：

特性	LoadAndScanFile	LoadFileOnly
行索引	✅ 完整构建	❌ 无
启动速度	极快 (<10ms)	更快 (<5ms)
内存使用	按需分块加载	按需分块加载
适用场景	大多数情况	特殊需求

4.3 状态查询

GetFileIsLoaded - 加载状态查询

```
FJIFFYLDG_API int GetFileIsLoaded(fjiffyldg_ptr fm);
```

功能：查询文件加载状态。

返回值：

- 0：加载正常，系统可用
- 非 0：加载错误代码（与 LoadAndScanFile 相同）

典型用途：

```
// 检查系统是否可用
int load_status = GetFileIsLoaded(fm);
if (load_status != 0) {
    fprintf(stderr, "系统不可用，错误： %d\n", load_status);
    return;
}
```

状态组合查询模式

```
// 完整状态检查函数
void check_system_status(fjiffyldg_ptr fm) {
    // 1. 检查加载状态（基础）
    int load_ok = GetFileIsLoaded(fm);
```

```
if (load_ok != 0) {
    printf("❌ 系统故障，错误码: %d\n", load_ok);
    return;
}

// 2. 检查扫描状态（高级）
long long scan_status = GetFileLineCount(fm);
if (scan_status > 0) {
    printf("✅ 系统完全就绪, %lld 行数据\n", scan_status);
} else if (scan_status == 0) {
    printf("⚠️ 系统就绪，但未开始扫描\n");
} else { // scan_status < 0
    printf("⌚ 扫描进行中...\n");
}
}
```

5. 行索引访问

5.1 行数查询

GetFileLineCount - 扫描状态查询

FJIFFYLDG_API long long GetFileLineCount(fjiffyldg_ptr fm);

功能：双重功能 - 查询扫描状态或获取总行数。

返回值语义：

返回值	含义	系统状态
> 0	扫描完成	返回确切总行数
= 0	扫描未开始	系统就绪但未扫描
< 0	扫描进行中	绝对值无意义，仅表示扫描中

使用模式：

```
long long status = GetFileLineCount(fm);

if (status > 0) {
    // 扫描完成，使用行数
    printf("文件共 %lld 行\n", status);
    process_by_lines(fm, status);
}
```

```

} else if (status == 0) {
    // 特殊状态：加载但未扫描
    printf("文件已加载，但未开始扫描\n");
    printf("可能是调用了 LoadFileOnly，或扫描被禁用\n");

} else { // status < 0
    // 扫描进行中
    printf("扫描进行中，请等待...\n");

    // 可选：等待扫描完成
    WaitFileScanTaskFinished(fm);
    status = GetFileLineCount(fm);

    if (status > 0) {
        printf("扫描完成，共 %lld 行\n", status);
    }
}
}

```

5.2 位置转换

GetFileLinePos

FJIFFYLDG_API long long GetFileLinePos(fjiffyldg_ptr fm, long long index);

功能：将行索引转换为字节偏移位置。

参数：

- **index**：行索引（0-based，第 0 行开始）

返回值：

- ≥ 0 ：该行起始字节位置
- < 0 ：索引无效或信息不可用

坐标系示例：

文件内容："Hello\nWorld\nEnd"

字节位置： 0 6 12

行索引： 0 1 2

伪代码示例：

GetFileLinePos(fm, 1) 返回 6

GetFileLinePos(fm, 2) 返回 12

GetFileLineIndex

FJIFFYLDG_API long long GetFileLineIndex(fjiffyldg_ptr fm, long long pos);

功能：将字节位置转换为行索引。

参数:

- **pos:** 字节偏移位置 (0-based)

返回值:

- ≥ 0 : 包含该字节的行索引
- < 0 : 位置无效或未找到

坐标系示例:

文件内容: "Hello\nWorld\nEnd"

字节位置: 0 6 12

行索引: 0 1 2

伪代码示例:

GetFileLineIndex(fm, 0~5) 返回 0

GetFileLineIndex(fm, 6~11) 返回 1

5.3 长度获取

GetFileLineLength

FJIFFYLDG_API long long GetFileLineLength(fjiffyldg_ptr fm, long long index);

功能: 获取指定行的字节长度 (不包含行终止符)。

参数:

- **index:** 行索引

返回值:

- ≥ 0 : 行内容长度 (字节)
- < 0 : 索引无效或信息不可用

重要说明:

- 长度不包括 \n、\r\n 等行终止符
- 空行长度为 0
- 可用于预分配缓冲区

使用示例:

```
// 安全读取一行内容
long long line_len = GetFileLineLength(fm, line_idx);
if (line_len >= 0) {
    // 预分配准确大小的缓冲区
    char* buffer = (char*)malloc(line_len + 1); // +1 for '\0'

    unsigned int read_len = line_len;
    const char* data = ReadFileData(fm,
                                    GetFileLinePos(fm, line_idx),
                                    &read_len);

    if (data && read_len == line_len) {
```

```
        memcpy(buffer, data, line_len);
        buffer[line_len] = '\0';
        printf("行 %lld: %s\n", line_idx, buffer);
    }

    free(buffer);
}
```

6. 数据读取操作

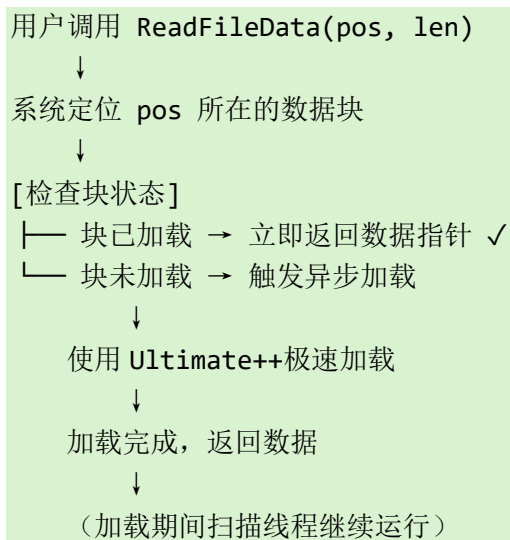
6.1 基本读取

ReadFileData - 智能触发加载

```
FJIFFYLDG_API const char* ReadFileData(fjiffyldg_ptr fm,
                                         long long pos,
                                         unsigned int *len);
```

功能：从指定位置读取文件数据，自动触发按需加载。

智能加载机制：



参数说明：

- pos: 起始字节位置 (0-based)
- len: 输入/输出参数
 - o 输入：期望读取长度 (0 表示使用默认大小)
 - o 输出：实际读取长度

返回值：

- 成功：指向数据的只读指针

- 失败: NULL

关键安全规则:

```
//  正确使用
const char* data = ReadFileData(fm, pos, &len);
if (data) {
    // 使用数据...
    process_data(data, len);
    // 注意：不需要也不应该释放 data!
}

//  危险操作
char* data = (char*)ReadFileData(fm, pos, &len); // 去掉 const 修饰
free(data); // 绝对禁止!
data[0] = 'X'; // 禁止修改!
```

性能特性:

- 首次访问：可能触发加载，但速度极快
- 局部性优化：可能预加载相邻块
- 零拷贝访问：直接返回内存映射数据
- 并发安全：加载不影响扫描线程

示例：

```
// 读取文件开头 1KB 数据
unsigned int want_len = 1024;
long long start_pos = 0;

const char* header = ReadFileData(fm, start_pos, &want_len);
if (header) {
    printf("成功读取 %u 字节头部数据\n", want_len);

    // 分析文件类型等
    if (want_len >= 3 && memcmp(header, "\xEF\xBB\xBF", 3) == 0) {
        printf("检测到 UTF-8 BOM 标记\n");
    }
}
```

6.2 行感知读取

ReadFileDataLLineCut - 行边界感知读取

[illegible]

```
long long *bpos,  
long long *epos,  
unsigned int *len);
```

功能：智能读取，自动处理行边界和长行截断。

设计特点：

- 自动截断超过 4KB 的超长行
- 保持行完整性（尽可能）
- 适合文本编辑器等行敏感应用

参数详解：

参数	方向	说明
index	输入/输出	起始行索引，返回结束行索引
bpos	输出	实际读取起始字节位置
epos	输出	理想的行对齐结束位置
len	输入/输出	期望/实际读取长度

使用模式：

```
// 模拟文本编辑器读取多行内容  
long long current_line = 0;  
long long bpos, epos;  
unsigned int want_len = 64 * 1024; // 期望读取 64KB  
int screen_lines = 0;  
  
while (screen_lines < 50) { // 填充 50 行屏幕  
    const char* screen_data = ReadFileDataLLLineCut(fm, &current_line,  
                                                    &bpos, &epos, &want_len);  
    if (!screen_data || want_len == 0) break;  
  
    // 计算这次读取了多少行  
    unsigned int actual_len = want_len;  
    long long lines_read = current_line - (screen_data ? 1 : 0);  
  
    printf("读取行 %lld-%lld, 字节范围[%lld,%lld), %u 字节\n",  
          lines_read, current_line, bpos, epos, actual_len);  
  
    screen_lines += (current_line - lines_read);  
  
    // 检查是否完整读取  
    if (actual_len < (epos - bpos)) {  
        // 未读完整，调整参数继续  
        want_len = epos - bpos - actual_len;  
    } else {  
        // 已读完整，重置为下一批
```

```

        want_len = 64 * 1024;
    }
}

```

ReadFileDataEndOfLine - 行内读取函数

```

FJIFFYLDG_API const char* ReadFileDataEndOfLine(fjiffyldg_ptr fm,
                                                long long index,
                                                long long pos,
                                                unsigned int *len);

```

功能

从指定行内的位置开始读取数据，确保只读取该行范围内的内容。

参数约束

- **index**: 必须是一个有效的行索引（0-based）
- **pos**: 必须满足: $\text{GetFileLinePos}(\text{fm}, \text{index}) \leq \text{pos} < \text{GetFileLinePos}(\text{fm}, \text{index}+1)$

与 **ReadFileData** 的区别

特性	ReadFileData	ReadFileDataEndOfLine
位置约束	任意位置	必须在该行内
读取范围	可能跨越多行	只读到该行结束
前置条件	无	需知道准确的行索引

适用场景

- 文本编辑器的行内操作
- CSV/TSV 文件的行内列提取
- 日志文件的特定行分析
- 需要严格行边界的数据处理

使用示例

```

// 读取第 5 行从第 10 字节开始的数据
long long line5_start = GetFileLinePos(fm, 5);
long long line5_len = GetFileLineLength(fm, 5);

if (line5_start >= 0 && line5_len > 10) {
    unsigned int want_len = 50;
    const char* data = ReadFileDataEndOfLine(fm, 5,
                                              line5_start + 10,
                                              &want_len);

    // data 只包含第 5 行 10-60 字节的数据，不会包含第 6 行
}

```

6.3 大文件内存映射

GetFileMappedHuge - 独立文件映射

```
FJIFFYLDG_API const char* GetFileMappedHuge(fjiffyldg_ptr fm,
                                             const char *fileName,
                                             long long *bufferSize);
```

功能：创建独立于 Load 系统的完整文件内存映射。

关键特性：

- 独立机制：与 LoadAndScanFile/LoadFileOnly 完全独立
- 零拷贝访问：直接内存映射，无额外复制
- 自动管理：Ultimate++ 自动保证不重复映射同一文件

自动资源管理

不重复映射保证

```
// Ultimate++ 自动保证：
// 1. 同一文件处理器(fm)不会重复映射同一文件
// 2. 新映射会自动替换旧映射
// 3. 旧映射资源自动释放，无泄漏风险

// 安全示例：
GetFileMappedHuge(fm, "data.bin", &size1); // 第一次映射
// ...使用 data1...

GetFileMappedHuge(fm, "data.bin", &size2); // 自动替换，无泄漏
// 现在只能使用 data2, data1 已自动失效
```

显式清理仍需要

```
// 规则：虽然自动管理，但显式清理仍是好习惯

// ✅ 推荐做法
const char* data = GetFileMappedHuge(fm, file, &size);
process_data(data, size);
ClearHugeBuffer(fm); // 显式清理，明确意图

// ⚠️ 可接受做法（依赖自动管理）
const char* data = GetFileMappedHuge(fm, file, &size);
process_data(data, size);
// 不调用 ClearHugeBuffer，稍后...
fjiffyldg_clear(fm); // 统一清理所有资源
```

使用模式

基础使用

```
long long size;
const char* data = GetFileMappedHuge(fm, "data.bin", &size);
```

```
if (data) {  
    process_mapped_data(data, size);  
    // Ultimate++保证：后续映射会自动管理  
}
```

连续映射同一文件

```
// 安全：Ultimate++自动处理  
GetFileMappedHuge(fm, "file.txt", &size1);  
// ...短暂使用...  
  
GetFileMappedHuge(fm, "file.txt", &size2); // 自动替换，无泄漏  
// 现在使用新的映射
```

映射不同文件

```
// 也需要清理，避免资源积累  
GetFileMappedHuge(fm, "file1.txt", &size1);  
process_file1(data1, size1);  
ClearHugeBuffer(fm); // 清理 file1 映射  
  
GetFileMappedHuge(fm, "file2.txt", &size2); // 新映射  
process_file2(data2, size2);  
ClearHugeBuffer(fm); // 清理 file2 映射
```

错误预防

指针失效警告

```
// 即使自动管理，指针仍需小心  
const char* old_data = GetFileMappedHuge(fm, file, &old_size);  
// ...保存 old_data 指针...  
  
GetFileMappedHuge(fm, file, &new_size); // 新映射  
// old_data 现在已失效！不要使用
```

总结

-  自动管理：Ultimate++保证同一文件不重复映射
-  显式清理：仍推荐 ClearHugeBuffer 明确意图
-  句柄清理：fjiffyldg_clear 会统一清理所有映射
-  指针安全：新映射使旧指针失效，需注意使用时机

7. 文件工具函数

7.1 文件操作

GetFileSizeByteCount - 快速文件大小查询

FJIFFYLDG_API long long GetFileSizeByteCount(const char *name);

功能：获取文件大小，无需加载文件。

特性：

- 直接查询文件系统
- 不占用文件处理器资源
- 可独立使用

返回值：

- ≥ 0 : 文件大小（字节）
- < 0 : 查询失败（文件不存在等）

使用示例：

```
// 在加载前检查文件大小
long long file_size = GetFileSizeByteCount("large_data.csv");

if (file_size < 0) {
    printf("文件不存在或无法访问\n");
    return;
}

// 根据大小决定处理策略
if (file_size > 1024 * 1024 * 100) { // >100MB
    printf("大文件警告: %.2f GB\n", file_size / (1024.0*1024.0*1024.0));
    printf("建议使用分批处理模式\n");
} else {
    printf("文件大小: %.2f MB\n", file_size / (1024.0*1024.0));
}
```

ToCloneFile - 高效文件复制

FJIFFYLDG_API int ToCloneFile(const char *oldFileName, const char *newFileName);

功能：复制文件，保持属性和权限。

返回值：

- 0 : 成功复制
- 1 : 源文件不符合预期
- < 0 : 严重错误

实现特点:

- 使用高效的分块复制
- 支持大文件 (>2GB)
- 保持文件元数据

ToSaveFile - 数据保存

```
FJIFFYLDG_API int ToSaveFile(const char *fileName, const char *buffer, long long len);
```

功能: 保存数据到新文件 (覆盖模式)。

ToAppendFile - 数据追加

```
FJIFFYLDG_API int ToAppendFile(const char *fileName, const char *buffer, long long len);
```

功能: 追加数据到现有文件。

ToConcatenateFile - 文件合并

```
FJIFFYLDG_API int ToConcatenateFile(const char *catFileName, const char *appendFileName);
```

功能: 将第二个文件内容追加到第一个文件。

使用场景:

```
// 合并多个日志文件
ToConcatenateFile("all_logs.txt", "log_202401.txt");
ToConcatenateFile("all_logs.txt", "log_202402.txt");
ToConcatenateFile("all_logs.txt", "log_202403.txt");

// 结果: all_logs.txt 包含三个文件内容
```

7.2 文本编码检测

统一返回值语义

所有三个检测函数的返回值遵循相同语义:

- 0: 整个缓冲区都符合检测的编码规则
- >0: 返回剩余长度, 表示剩下的可能不符合编码规则

函数详解

CheckTextASCII - ASCII 检测

```
FJIFFYLDG_API unsigned int CheckTextASCII(const char *text, unsigned int len);
```

功能：检测缓冲区是否为纯 ASCII。

返回值：

- 0: len 个字节全部是 ASCII 字符
- N ($0 < N \leq \text{len}$): 前 len-N 个字节是 ASCII, 剩余 N 个字节可能包含非 ASCII

示例：

```
const char* str = "Hello 世界"; // 共 12 字节: "Hello " (6) + "世界" (6)
unsigned int result = CheckTextASCII(str, 12);
// 返回 6, 表示:
// - 前 6 字节 ("Hello ") 是 ASCII
// - 剩余 6 字节 ("世界"部分) 包含非 ASCII
```

CheckWholeTextUtf8 - 完整 UTF-8 验证

```
FJIFFYLDG_API unsigned int CheckWholeTextUtf8(const char *text, unsigned int len);
```

功能：严格验证整个缓冲区是否为完整有效的 UTF-8 序列。

返回值：

- 0: 整个缓冲区是完整有效的 UTF-8
- N ($0 < N \leq \text{len}$): 前 len-N 个字节是有效 UTF-8, 剩余 N 个字节存在编码错误

重要限制：缓冲区必须包含完整 UTF-8 字符，不能有截断的多字节字符。

示例：

```
const char* good = "Hello 世界"; // 完整 UTF-8
unsigned int r1 = CheckWholeTextUtf8(good, 12); // 返回 0

const char* bad = "Hello 世\xff"; // 非法字节
unsigned int r2 = CheckWholeTextUtf8(bad, 10); // 返回 1
```

CheckExtractTextUtf8 - 采样 UTF-8 检测

```
FJIFFYLDG_API unsigned int CheckExtractTextUtf8(const char *text, unsigned int len);
```

功能：检测缓冲区是否可能是 UTF-8 编码，适用于可能被截断的文本。

返回值：

- 0: 检测部分看起来像 UTF-8
- N ($0 < N \leq \text{len}$): 前 len-N 个字节像 UTF-8, 剩余 N 个字节明确不是 UTF-8

设计特点：对截断字符友好，不会因为文本被截断而报错。（文本长度 ≥ 10 字节时更可靠）

示例：

```
// 截断的 UTF-8 文本 ("界"字需要 3 字节, 这里只有 1 字节)
const char* truncated = "Hello 世\xff"; // 只有"界"的前 1 字节
unsigned int result = CheckExtractTextUtf8(truncated, 10);
// 可能返回 0, 因为截断处停止检查, 不视为错误
```

使用对比

相同点

- 返回值语义一致：**0**=完全符合，**>0**=剩余长度
- 都从缓冲区开始逐字节检查
- 发现第一个不符合规则的位置即返回

不同点

函数	检测规则	对截断文本	典型用途
CheckTextASCII	ASCII 字符集	友好	快速 ASCII 检测
CheckWholeTextUtf8	完整 UTF-8 语法	不友好	严格验证
CheckExtractTextUtf8	宽松 UTF-8 检测	友好	编码推测，抽样验证

示例：

```
// 获取缓冲区中符合编码的有效部分长度
unsigned int get_valid_utf8_length(const char* text, unsigned int len) {
    unsigned int result = CheckWholeTextUtf8(text, len);
    if (result == 0) {
        return len; // 全部有效
    } else {
        return len - result; // 有效部分长度
    }
}
```

注意事项

1. 所有函数都检查整个缓冲区，不会跳过起始位置
2. 返回值剩余长度可能等于输入长度，表示第一个字节就不符合
3. 对于空缓冲区（len=0），所有函数都应返回 0

7.3 编码字符统计

GetUtf8TextCharCount - UTF-8 字符统计

```
FJIFFYLDG_API unsigned int GetUtf8TextCharCount(const char * *text, unsigned int len);
```

功能：统计 UTF-8 字符数（非字节数）。

参数特性：

- **text**: 输入/输出参数，自动推进指针
- **len**: 最大检查字节数

使用示例：

```
const char* utf8_str = "UTF-8 示例: 🍌 表情符号";
const char* cursor = utf8_str;
```

```
unsigned int char_count = 0;
unsigned int total_bytes = strlen(utf8_str);

while (total_bytes > 0) {
    unsigned int chars_in_segment = GetUtf8TextCharCount(&cursor, total_bytes);
    char_count += chars_in_segment;

    // 更新剩余字节数
    unsigned int bytes_processed = cursor - utf8_str;
    total_bytes = strlen(utf8_str) - bytes_processed;

    printf("段字符数: %u, 累计: %u\n", chars_in_segment, char_count);
}

printf("总字符数: %u, 总字节数: %zu\n",
       char_count, strlen(utf8_str));
```

8. C++ 接口封装

8.1 C++ 类接口

Fjiffyldg::Fjiffyldg 类定义

```
namespace Fjiffyldg {
    class FJIFFYLDG_API Fjiffyldg {
    public:
        // 构造函数: 自动创建底层资源
        Fjiffyldg();

        // 移动语义支持
        Fjiffyldg(Fjiffyldg &&) = default;
        Fjiffyldg& operator=(Fjiffyldg &&) = default;

        // 复制禁用 (资源句柄唯一性)
        Fjiffyldg(const Fjiffyldg &) = delete;
        Fjiffyldg& operator=(const Fjiffyldg &) = delete;

        // 获取底层 C 句柄 (用于调用 C API)
        fjiffyldg_ptr GetFjiffyldgHandle();
    };
}
```

```
// 析构函数：自动清理所有资源
~Fjiffyldg();

private:
    struct Impl;
    std::unique_ptr<Impl> pimpl; // PIMPL 模式，隐藏实现
};
}
```

8.2 C++使用示例

基本用法

```
#include <Fjiffyldg/Fjiffyldg.h>
#include <iostream>
#include <string>

int main() {
    try {
        // 1. 创建 C++对象（自动资源管理）
        Fjiffyldg::Fjiffyldg file_processor;

        // 2. 加载文件（使用 C API，但通过 C++对象管理）
        int load_result = LoadAndScanFile(
            file_processor.GetFjiffyldgHandle(),
            "data.txt"
        );

        if (load_result != 0) {
            std::cerr << "加载失败，错误码： " << load_result << std::endl;
            return 1;
        }

        // 3. 使用其他功能
        long long line_count = GetFileLineCount(
            file_processor.GetFjiffyldgHandle()
        );

        std::cout << "文件加载成功" << std::endl;

        if (line_count > 0) {
```

```

        std::cout << "总行数: " << line_count << std::endl;
    } else if (line_count < 0) {
        std::cout << "扫描进行中..." << std::endl;
        WaitFileScanTaskFinished(file_processor.GetFjiffyldgHandle());
        line_count = GetFileLineCount(file_processor.GetFjiffyldgHandle());
        std::cout << "扫描完成, 行数: " << line_count << std::endl;
    }

    // 4. 对象离开作用域时自动清理
    // ~Fjiffyldg() 自动调用 fjiffyldg_clear()

} catch (const std::exception& e) {
    std::cerr << "异常: " << e.what() << std::endl;
    return 1;
}

return 0;
}

```

9. 高级功能

9.1 异步扫描控制

WaitFileScanTaskFinished

FJIFFYLDG_API void WaitFileScanTaskFinished(fjiffyldg_ptr fm);

功能: 等待后台扫描任务完成。

使用场景:

1. 需要完整行索引时

```

LoadAndScanFile(fm, "large.log");
// ... 其他初始化工作 ...
WaitFileScanTaskFinished(fm); // 确保索引就绪
long long total = GetFileLineCount(fm);

```

2. 批量处理前

```

// 启动多个文件扫描
for (int i = 0; i < file_count; i++) {
    LoadAndScanFile(handles[i], files[i]);
}

```

```
// 等待所有扫描完成
for (int i = 0; i < file_count; i++) {
    WaitFileScanTaskFinished(handles[i]);
}
```

```
// 现在所有文件索引都就绪了
```

3. 性能测试时

```
LoadAndScanFile(fm, "test.txt");
WaitFileScanTaskFinished(fm); // 确保扫描完成
auto end_time = std::chrono::high_resolution_clock::now();
// 计算纯扫描时间
```

9.2 重新扫描功能

RestartScanFile

```
FJIFFYLDG_API void RestartScanFile(fjiffyldg_ptr fm, const char *name,
                                   long long offset, int utf);
```

功能：重新扫描文件，可指定起始位置和编码。

参数详解：

参数	说明
name	文件路径（可重新指定不同文件）
offset	扫描起始字节偏移（通常为 0）
utf	编码模式：0=默认（ASCII 兼容）；-1=自动检测 BOM；1=UTF-16LE；2=UTF-16BE；3=UTF-32LE；4=UTF-32BE

应用场景：

```
// 场景 1：重新扫描（文件内容改变）
LoadAndScanFile(fm, "log.txt");
// ... 文件被其他进程修改 ...
RestartScanFile(fm, "log.txt", 0, 0); // 重新扫描

// 场景 2：以不同编码重新扫描
LoadAndScanFile(fm, "unknown.txt");
// 发现编码不正确
RestartScanFile(fm, "unknown.txt", 0, -1); // 尝试自动检测

// 场景 3：从中间开始扫描（跳过头部）
RestartScanFile(fm, "huge.bin", 1024*1024, 0); // 从 1MB 处开始
```

9.3 性能优化指南

访问模式优化

顺序访问（最优性能）

```
// 连续行访问，触发智能预读
for (long long i = 0; i < total_lines; i++) {
    unsigned int len = 0;
    const char* line = ReadFileData(fm,
                                    GetFileLinePos(fm, i),
                                    &len);

    // 处理...
}
```

随机访问（性能良好）

```
// 随机行访问，按需加载相应块
long long random_lines[] = {10, 100, 500, 1000};
for (int i = 0; i < 4; i++) {
    long long pos = GetFileLinePos(fm, random_lines[i]);
    if (pos >= 0) {
        unsigned int len = 0;
        const char* data = ReadFileData(fm, pos, &len);
        // 处理...
    }
}
```

批量访问建议

```
// 1. 先收集所有需要的位置
std::vector<long long> positions;
for (auto& query : queries) {
    long long pos = GetFileLinePos(fm, query.line);
    if (pos >= 0) positions.push_back(pos);
}

// 2. 按位置排序（提升缓存命中）
std::sort(positions.begin(), positions.end());

// 3. 顺序读取
for (auto pos : positions) {
    unsigned int len = 1024;
    const char* data = ReadFileData(fm, pos, &len);
    // 处理...
}
```

```
}
```

内存管理策略

大文件处理

```
// 策略 1: 使用内存映射 (适合<2GB 文件)
long long size;
const char* mapped = GetFileMappedHuge(fm, "large.bin", &size);
if (mapped) {
    process_mapped_file(mapped, size);
    ClearHugeBuffer(fm); // 及时释放
}

// 策略 2: 分块处理 (适合任意大小)
unsigned int block_size = 1024 * 1024; // 1MB 块
for (long long offset = 0; offset < file_size; offset += block_size) {
    unsigned int len = block_size;
    const char* block = ReadFileData(fm, offset, &len);
    process_block(block, len);
}
```

多文件处理

```
// 为每个文件创建独立处理器
std::vector<fjiffyldg_ptr> processors;

for (auto& filename : filenames) {
    fjiffyldg_ptr fm = fjiffyldg_create();
    if (fm && LoadAndScanFile(fm, filename.c_str()) == 0) {
        processors.push_back(fm);
    }
}

// 并发处理多个文件
#pragma omp parallel for
for (size_t i = 0; i < processors.size(); i++) {
    process_single_file(processors[i]);
}

// 清理所有处理器
for (auto fm : processors) {
    fjiffyldg_clear(fm);
}
```

附录 A：API 速查表

A.1 核心管理与生命周期

函数	功能	线程安全	资源管理
fjiffyldg_create()	创建文件处理器实例	是	返回的句柄需配对清理
fjiffyldg_clear()	释放所有资源（包括映射）	否（仅对自身）	必须与 fjiffyldg_create() 配对
ClearHugeBuffer()	清理映射	是	清理映射资源
LoadAndScanFile()	启动扫描并发系统	否	返回 0 仅表示初始化成功
LoadFileOnly()	仅加载文件内容	否	简化模式，无行索引

A.2 状态查询与监控

函数	返回类型	含义	典型值解释
GetFileIsLoaded()	int	加载系统状态	0=正常，非 0=错误代码
GetFileLineCount()	long long	扫描系统状态	>0=总行数，0=未开始，<0=进行中
GetFileSizeByteCount()	long long	文件大小（字节）	≥0=大小，<0=错误
WaitFileScanTaskFinished()	void	等待扫描完成	无返回值，阻塞调用

A.3 数据读取与访问

函数	参数约束	读取边界	适用场景
ReadFileData()	任意位置	可能跨行	通用数据读取
ReadFileDataEndOfLine()	必须在行内	只到行尾	行内精确操作
ReadFileDataLineCut()	自动行感知	行边界智能处理	文本编辑器
GetFileMappedHuge()	独立于 Load 系统	完整文件	一次性完整访问

A.4 行索引与位置转换

函数	方向	时间复杂度	查找过程	实际工程性能
GetFileLinePos()	行→位置	$O(\log n) + O(m)$	1. 二分找块 $O(\log n)$ 2. 块内查找 $O(m)$	前 100 万行： $O(1)$ 超 100 万行： $\sim O(\log n + m)$
GetFileLineIndex()	位置→行	$O(\log n) + O(m)$	1. 二分找块 $O(\log n)$ 2. 块内定位 $O(m)$	依赖位置分布

GetFileLineLength()	计算行长	$2 \times [O(\log n) + O(m)]$	两 GetFileLinePos() 调用	前 100 万行： $O(1)$
---------------------	------	-------------------------------	--------------------------	---------------------

A.5 文件操作工具

函数	功能	返回值	原子性
ToCloneFile()	复制文件	0=成功	是
ToSaveFile()	保存文件	0=成功	是
ToAppendFile()	追加内容	0=成功	是
ToConcatenateFile()	合并文件	0=成功	是

A.6 文本编码检测

函数	文本要求	返回值语义	目的
CheckTextASCII()	任意文本	0=全 ASCII, >0=剩余长度	快速 ASCII 检测
CheckWholeTextUtf8()	完整 UTF-8 文本	0=有效, >0=剩余长度	严格验证
CheckExtractTextUtf8()	任意文本 (含截断)	0=可能 UTF-8, >0=剩余长度	截断文本检测
GetUtf8TextCharCount()	UTF-8 文本	字符数 (非字节数)	字符统计

A.7 扫描控制与编码设置

函数	参数说明	主要用途	并发影响
RestartScanFile()	offset= 起始偏移, utf=编码模式	重新扫描/切换编码	停止当前扫描
编码模式常量	0=默认, -1=BOM 自动检测, 1-4=UTF 编码	指定扫描编码	影响扫描结果

A.8 C++ 接口封装

类/成员	功能	资源管理	异常安全
Fjiffyldg::Fjiffyldg	C++ 主类	RAII 自动管理	是
GetFjiffyldgHandle()	获取底层 C 句柄	由类生命周期管理	否
移动构造/赋值	支持移动语义	资源转移	是
复制构造/赋值	被禁用	-	-

A.9 错误代码速查

代码	含义	常见原因	处理建议
0	成功	-	继续正常流程
-1	文件不存在	路径错误、权限不足	检查路径和权限
1	文件不可访问	文件被锁定、权限问题	检查文件状态
2	文件流错误	文件损坏、磁盘错误	验证文件完整性
3	内存映射失败	内存不足、文件太大	减少并发或使用分块

A.10 性能特征速查

核心操作性能

操作	典型耗时	影响因素	优化建议
启动响应	<10ms	首块加载速度	使用默认设置
数据块加载	<5ms/块	磁盘速度、块大小	顺序访问有预读优化
随机行访问	<2ms	索引是否就绪	等待扫描完成可提速

行访问性能特征表

场景	查找机制	时间复杂度	典型耗时	出现频率
前 100 万行	直接数组索引	$O(1)$	<1 μs	>95%
超 100 万行（在分块内）	1. 二分找块 $O(\log n)$; 2. 块内查找 $O(m)$	$O(\log n + m)$	2-10 μs	~4%
超大文件（多块）	1. 二分找块 $O(\log n)$; 2. 块内查找 $O(m)$	$O(\log n + m)$	10-50 μs	<1%

参数说明：

- n = 实际块数（通常远小于文件行数）
- m = 块内查找开销（每个块有自己的索引结构）

扫描与索引构建性能

操作	性能特征	对用户操作影响	建议策略
行索引构建	取决于文件大小	零阻塞（后台执行）	无需等待，立即操作
扫描期间访问	正常速度	可同时读写文件	设计为即时响应应用
完整索引就绪	一次性等待	之后所有访问最优	分析工具可等待完成

内存使用特征

使用模式	内存占用	增长特征	适用场景
分块加载	按需加载块大小	与访问模式相关	大文件、内存受限
完整映射	整个文件大小	一次性分配	中小编文件、高性能需求
索引内存	分层可控增长	前 100 万行线性，之后平缓	千万行文件可管理

编码检测性能

函数	检测速度	文本要求	适用场景
----	------	------	------

CheckTextASCII()	极快 (O(n))	任意文本	快速 ASCII 验证
CheckWholeTextUtf8()	快 (O(n))	完整 UTF-8 文本	配置/数据文件验证
CheckExtractTextUtf8()	快 (O(n))	任意文本 (含截断)	日志/流式数据处理

并发性能特征

并发场景	性能表现	线程安全策略	建议用法
单句柄多线程读	需外部同步	非线程安全	每个线程独立实例
多句柄同文件	良好	独立资源管理	并行处理的不同阶段
扫描与读取并发	最佳 (零阻塞)	架构设计保障	实时处理应用

文件操作性能

操作	原子性	典型性能	适用场景
ToCloneFile()	是	接近系统复制速度	大文件分块复制
ToSaveFile()	是	取决于写入大小	覆盖模式
ToAppendFile()	是	快速追加	日志记录场景
ToConcatenateFile()	是	流式合并	避免超大文件合并

性能边界认知

理论边界

- 最大文件：8EB (理论值，实际受 OS 限制)
- 最大行数：离散块索引支持千万级行数
- 内存使用：前 100 万行线性增长，之后平缓

A.11 编码模式常量值

值	编码	说明
0	默认模式	ASCII 控制符兼容
-1	自动检测	基于文件开头 BOM
1	UTF-16LE	小端序 UTF-16
2	UTF-16BE	大端序 UTF-16
3	UTF-32LE	小端序 UTF-32
4	UTF-32BE	大端序 UTF-32

使用提示：

1. 状态查询：操作前检查 GetFileIsLoaded() 确保系统正常
2. 资源管理：C 语言必须配对清理，C++ 依赖 RAII
3. 并发注意：单个句柄非线程安全，多线程需外部同步
4. 性能调优：根据访问模式选择合适 API (顺序 vs 随机)

附录 B：版本历史

V1.0 (2025 年 12 月 31 日)

- 初始版本发布
- 支持文件行索引构建
- 提供内存映射文件访问
- 实现多编码文本兼容
- 包含完整文件操作工具集

已知限制

1. 单个文件处理器不支持同时加载多个文件
2. 64 位系统适配，32 位需要特别处理
3. 某些边缘编码检测可能不准确

技术支持与反馈

关于名称的说明：

本软件简称为 **Fjiffyldg**，为方便技术实现，接口头文件名和库文件名采用小写，GitHub 仓库使用小写 **fjiffyldg** 以便 URL 访问。

项目主页

<https://github.com/ceepuka/fjiffyldg>

问题反馈

<https://github.com/ceepuka/fjiffyldg/issues>

邮件联系

2641451398@qq.com

版权声明

版权所有 © 2025 Du Jie (@ceepuka)。保留所有权利。

文档版本：V1.0.1

最后更新：2026 年 1 月